

(OBJECT ORIENTED PROGRAMMING) ★

OOPs is a way of thinking. It helps us to model real-world entities as Objects and solve complex problems easily.

Building better software using Objects & Classes

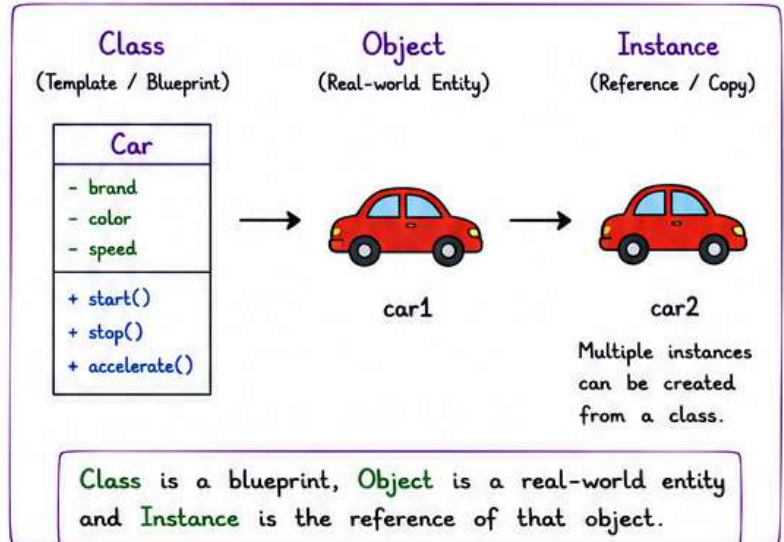
1. What is OOP?

OOP (Object Oriented Programming) is a programming paradigm based on the concept of "Objects", which can contain data (attributes) and code (methods).

2. Why OOP?

- To manage complexity in large software projects.
- To improve code reusability and maintainability.
- To provide a clear structure for the program.
- To map real-world entities naturally.

7. Class → Object → Instance



3. Procedural Programming vs Object Oriented Programming

Procedural Programming	Object Oriented Programming
• Focus on procedures / functions.	• Focus on objects and classes.
• Program is divided into functions.	• Program is divided into objects.
• Data and functions are separate.	• Data and functions are bundled together.
• Less secure (global data access).	• More secure (data hiding).
• Difficult to maintain in large projects.	• Easy to maintain and extend.
• Example: C language	• Example: Java, C++, Python, C#

4. Class vs Object

Class:

A class is a blueprint or template that defines the properties and behaviors of an object.

Object:

An object is a real-world entity created from a class. It has state (data) and behavior (methods).

Example:

Class → Car (blueprint)
Object → MyCar (real car)

5. Attributes & Methods

- **Attributes:** Variables that represent the state of an object.
Example: brand, color, speed
- **Methods:** Functions that represent the behavior of an object.
Example: start(), stop(), accelerate()

6. Real-world Examples

Student	Bank Account	Mobile Phone	Book
Attributes: name, rollNo, age	Attributes: accNo, balance, type	Attributes: brand, model, storage	Attributes: title, author, price
Methods: study(), attendClass()	Methods: deposit(), withdraw()	Methods: call(), message(), installApp()	Methods: open(), read(), close()

8. Benefits of OOP

- ✓ **Modularity:** Code is divided into independent objects.
- ✓ **Reusability:** Objects and classes can be reused.
- ✓ **Maintainability:** Easy to modify and maintain code.
- ✓ **Scalability:** Easy to extend and scale large projects.
- ✓ **Security:** Data hiding protects data from outside access.
- ✓ **Flexibility:** Supports polymorphism and dynamic behavior.

★ Interview Tip

- ★ OOPs is not just about classes and objects.
- ★ It is about solving real-world problems efficiently.
- ★ Always think: What are the objects in this problem? What are their properties and actions?



"The goal of OOPs is not to write more code, but to organize code better."





ENCAPSULATION



Encapsulation is the mechanism of wrapping data (variables) and code (methods) together as a **single unit** and **restricting direct access** to some of the object's components.



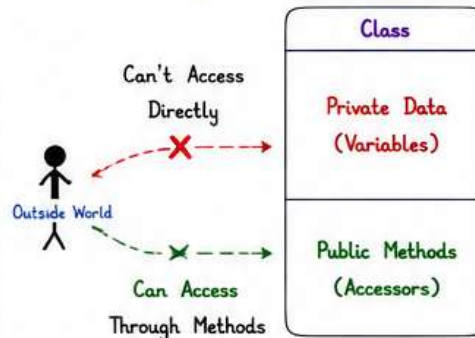
Interview Tip

Encapsulation is all about "Data Hiding". It increases the **security** and **control** over the data.

1. Key Points

- **Data hiding** is achieved using **access modifiers**.
- The **internal state** of an object is hidden from the outside world.
- Access to data is provided through **public methods** (**getters** and **setters**).
- It **protects** data from unauthorized access and modification.

2. Data Hiding



Benefit

- Security
- Controlled Access
- Data Validation
- Maintainability

3. Access Modifiers in Java

Modifier	Within Class	Within Package	Outside Package (Non-subclass)	Outside Package (Subclass)
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✗	✓
public	✓	✓	✓	✓

✓ = Accessible

✗ = Not Accessible

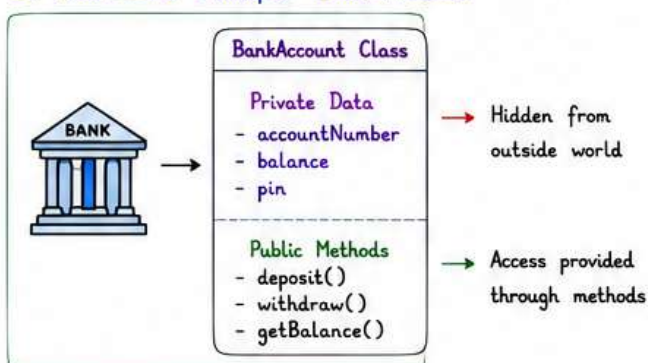
4. Getters and Setters

- **Getter**: Used to read the value of a private variable.
- **Setter**: Used to modify the value of a private variable.

Why use Getters and Setters?

- To validate data before assigning.
- To make variables read-only or write-only.
- To maintain the flexibility of changing implementation later.

5. Real-world Example: Bank Account



7. Advantages of Encapsulation

- ✓ Protects data from misuse.
- ✓ Provides controlled access to data.
- ✓ Makes code more modular and easy to maintain.
- ✓ Allows data validation before storing.
- ✓ Increases flexibility and reusability of code.

6. Java Example

```
class BankAccount {
    private String accountNumber;
    private double balance;
    private int pin;
    // private data

    // Getter
    public double getBalance() {
        return balance;
    }

    // Setter
    public void deposit(double amount) {
        if(amount > 0) {
            balance = balance + amount;
        }
    }

    public boolean withdraw(double amount, int pin) {
        if(this.pin == pin && amount > 0 && amount <= balance) {
            balance = balance - amount;
            return true;
        }
        return false;
    }
}
```



Common Interview Questions

1. What is Encapsulation?
2. How is data hiding achieved in Java?

3. Why do we use getters and setters?
4. What are the advantages of Encapsulation?



= ABSTRACTION =

Abstraction is the process of **hiding** the internal implementation details and showing only the **essential features** of an object.

It helps in reducing complexity and increasing security.



Interview Tip

Abstraction is about showing "what" an object does, and hiding "how" it does it.

1. What is Abstraction?

- It is one of the key features of OOP.
- It focuses on **what** an object **does** instead of **how** it does it.
- It hides unnecessary details and shows only the required functionality.
- Achieved using **Abstract Classes** and **Interfaces**.

2. Real-world Example - Car

You drive a Car.

You know:

- Start
- Stop
- Accelerate
- Brake



You don't know:

- How engine works
- How fuel burns
- How gears change
- Internal electronics

This is Abstraction - You use the car by knowing only important features, not the internal working.

3. Abstract Class vs Interface

Feature	Abstract Class	Interface
Definition	A class that cannot be instantiated and may contain abstract methods (without body) and concrete methods.	A completely abstract type that contains only abstract methods (before Java 8).
Methods	Can have both abstract and concrete methods.	Only abstract methods (default & static allowed in Java 8+).
Variables	Can have instance variables.	Can have only public static final variables (constant).
Constructor	Can have constructors.	Cannot have constructors.
Access Modifiers	Can have any access modifiers.	Methods are public by default.
Inheritance	A class can extend only one abstract class.	A class can implement multiple interfaces.
Keyword	abstract	interface
Example Use	When we want to share common code among related classes.	When we want to define a contract/behavior that classes must follow.

4. Abstract Class Example (Java)

```
abstract class Shape {           // abstract class
    abstract void draw();        // abstract method

    void display() {             // concrete method
        System.out.println("This is a shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

// Usage
Shape s1 = new Circle();
s1.draw();           // Drawing Circle
s1.display();        // This is a shape
```

Note

We cannot create object of abstract class.

We can create object of its subclass.



5. Interface Example (Java)

```
interface Drawable {             // interface
    void draw();                 // abstract method
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Square implements Drawable {
    public void draw() {
        System.out.println("Drawing Square");
    }
}

// Usage
Drawable d1 = new Circle();
d1.draw();           // Drawing Circle
```

Note

From Java 8, interface can have default and static methods.

From Java 9, interfaces can have private methods.



6. Key Takeaway

- ✓ Abstraction hides the internal details and shows only essential features.
- ✓ It reduces complexity and improves security.
- ✓ Use **Abstract Class** when you want to share code and state.
- ✓ Use **Interface** when you want to define a contract/behavior.

? Common Interview Questions

1. What is **Abstraction** in OOP?
2. How is **Abstraction** achieved in Java?
3. Difference between **Abstract Class** and **Interface**?
4. When to use **Abstract Class** and **Interface**?



"Abstraction lets you focus on the **important things** without worrying about the unnecessary details."





INHERITANCE



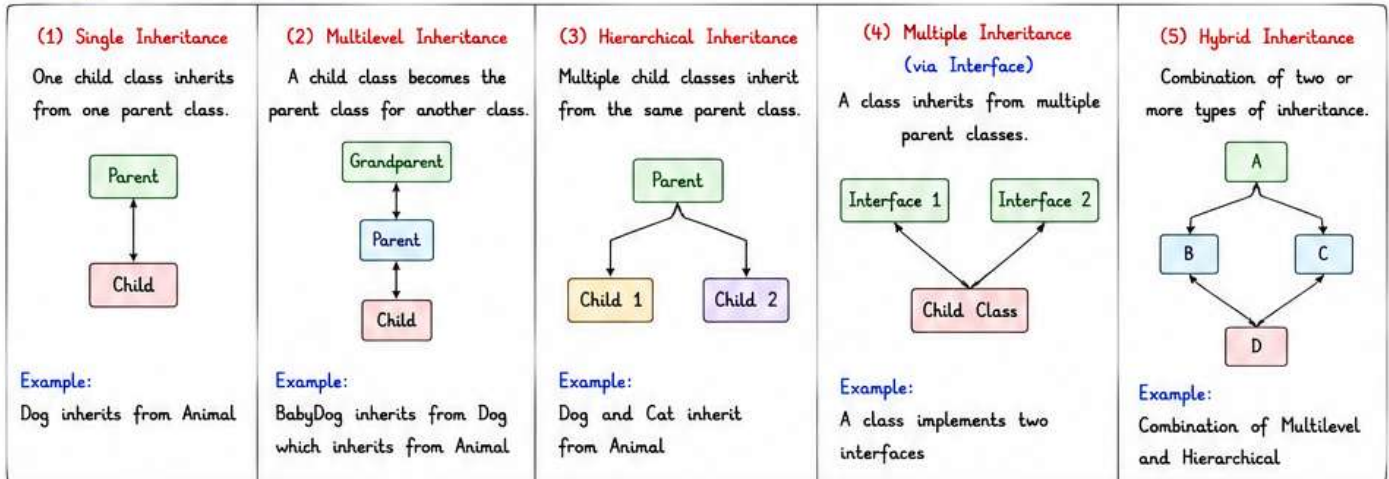
Inheritance is a mechanism where a new class (child / subclass) acquires the properties and behaviors (methods) of an existing class (parent / superclass). It helps in **code reusability** and establishes a **natural hierarchy**.



Interview Tip

Inheritance represents the "IS-A" relationship. It promotes code reusability and extensibility.

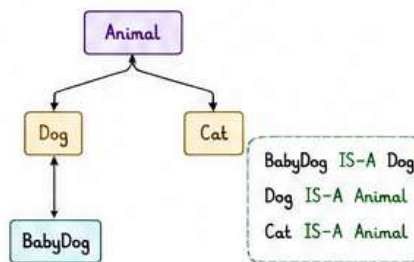
1. Types of Inheritance



2. IS-A Relationship

Inheritance represents an IS-A relationship.

e.g., A Dog IS-A Animal
A Car IS-A Vehicle



3. super Keyword

- The super keyword is used to refer to the immediate parent class.

- It can be used to:
 - Access parent class class variables
 - Call methods
 - Call parent class constructor

```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    void sound() {
        super.sound(); // calls parent class method
        System.out.println("Bark");
    }
}
          
```

4. Method Overriding

- When a subclass provides a specific implementation of a method already defined in its parent class, it is called **Method Overriding**.
- Achieved by having the same method signature.
- Used for Runtime Polymorphism.

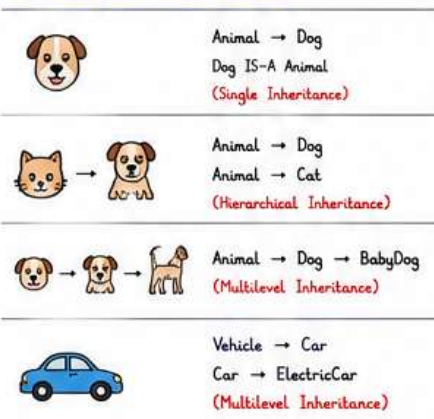
```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
          
```

5. Key Points

- ✓ Inheritance promotes code reusability.
- ✓ Subclass inherits non-private members of superclass.
- ✓ Private members are not inherited (but can be accessed indirectly through methods).
- ✓ Constructor is not inherited.
- ✓ Use **extends** keyword to inherit a class.

6. Example: Real-world



7. Inheritance in Java (Syntax)

```

// Parent / Superclass
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}
// Child / Subclass
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
// Usage
Dog d = new Dog();
d.eat(); // inherited method
d.bark(); // own method
          
```

8. Quick Summary

Relationship	IS-A
Keyword	extends
Reusability	✓
Constructor	Not inherited
Access	Non-private members are inherited
Polymorphism	Supported (Method Overriding)



Key Takeaway: Inheritance helps in building a hierarchy of classes and reusing code effectively.

Always design your classes in a way that "IS-A" relationship makes sense!





POLYMORPHISM



Polymorphism allows objects of different classes to be treated as objects of a common type. It lets the **same method name** behave differently based on the **object** that calls it.



Interview Tip

Polymorphism means "Many Forms".
One interface, multiple implementations.

1. Types of Polymorphism

(A) Compile-time Polymorphism (Static Polymorphism)	(B) Runtime Polymorphism (Dynamic Polymorphism)
- Achieved by Method Overloading. - Resolved at compile time. - Also called Static Binding. - Improves code readability.	- Achieved by Method Overriding. - Resolved at runtime. - Also called Dynamic Binding. - Enables code flexibility.

2. Compile-time vs Runtime Polymorphism

Feature	Compile-time (Static)	Runtime (Dynamic)
Also called	Static Binding	Dynamic Binding
Resolved at	Compile Time	Runtime
Achieved by	Method Overloading	Method Overloading
Inheritance	Not Required	Required
Keyword	-	@Override
Performance	Faster	Slightly Slower
Example	add(int, int), add(int, int, int)	Dog sound(), Cat sound()

3. Method Overloading (Compile-time Polymorphism)

Same class, same method name, **different parameters** (number, type or order).

How it works?

- Compiler decides which method to call based on parameters.
- Which method is called is decided at **compile time**.

```
class MathUtil {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Same method name → Different parameters → Compile-time decision

4. Method Overriding (Runtime Polymorphism)

Subclass provides a **specific** implementation of a method already defined in its parent class.

How it works?

- JVM decides which method to call based on object at runtime.
- Which method is called is decided at **runtime**.

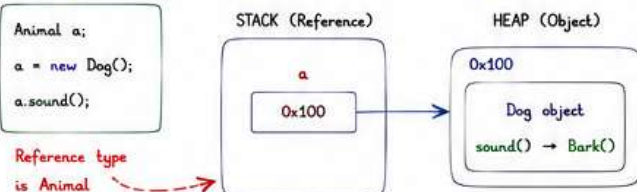
```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}
```

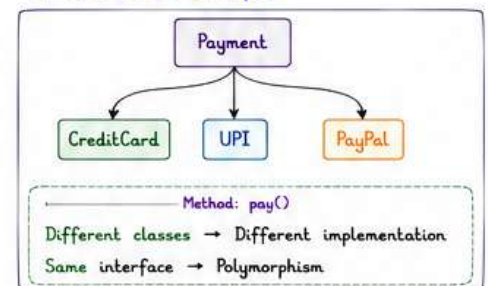
Same method name → Different implementation → Runtime decision

5. Dynamic Method Dispatch (How Runtime Polymorphism)



JVM at Runtime
Looks at the actual object (Dog) and calls Dog's sound() method.

6. Real-world Example



7. Polymorphism in Interfaces (100% Runtime)

```
interface Payment {
    void pay();
}

class CreditCard implements Payment {
    public void pay() {
        System.out.println("Paid using Credit Card");
    }
}

class UPI implements Payment {
    public void pay() {
        System.out.println("Paid using UPI");
    }
}
```

```
Payment p;
p = new CreditCard();
p.pay(); // Paid using Credit Card

p = new UPI();
p.pay(); // Paid using UPI
```

★ Interface reference can point to any object that implements that interface.

8. Key Takeaways

- ✓ Polymorphism increases flexibility and extensibility.
- ✓ Method **Overloading** → Compile-time Polymorphism.
- ✓ Method **Overriding** → Runtime Polymorphism.
- ✓ Use @Override for method overriding.
- ✓ Always program to the **interface**, not the implementation.
- ✓ Polymorphism helps in writing clean, maintainable and scalable code.

9. Quick Comparison (Overloading vs Overriding)

Feature	Method Overloading	Method Overriding
Polymorphism Type	Compile-time (Static)	Runtime (Dynamic)
Method Name	Same	Same
Parameters	Different	Same
Return Type	Can be different	Must be same (or covariant)
Inheritance Required?	No	Yes
Access Modifier	Any	Cannot be more restrictive
Keyword	-	@Override

Common Interview Questions

1. What is Polymorphism in OOP?
2. Explain the types of Polymorphism.
3. Difference between Overloading and Overriding?
4. What is Dynamic Method Dispatch?
5. Why is Runtime Polymorphism slower?



"Polymorphism is the heart of OOP - One interface, many implementations."



ASSOCIATION, AGGREGATION & COMPOSITION

These are the ways one class (object) relates to another class (object).

They represent **HAS-A** relationship.

The difference is in the **strength** of their relationship and **lifecycle** dependency.



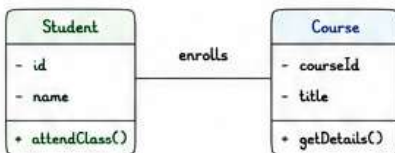
Interview Tip

These Relationships represent
"**HAS-A**" relationship.

They help in modeling real-world
connections between objects.

1. ASSOCIATION (Weak Relationship)

- A general relationship between two classes.
- Both classes can exist independently.
- Change in one object does not affect the other object.
- Represented by a simple line.



Real-world Example:

Student enrolls in a Course.

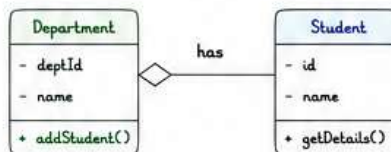
(Student can exist without Course,
and Course can exist without Student)

UML Notation:



2. AGGREGATION (Weak HAS-A)

- A specialized form of association.
- Represents **HAS-A** relationship.
- Child object can exist independently of the parent.
- If parent is destroyed, child is **NOT** destroyed.
- Represented by a hollow diamond.

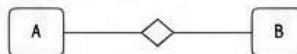


Real-world Example:

Department has Students.

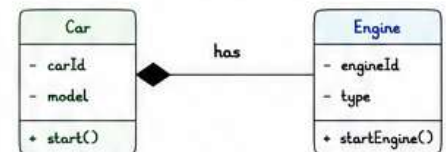
(Students can exist even if Department
is removed from the system)

UML Notation:



3. COMPOSITION (Strong HAS-A)

- A **strong** form of aggregation.
- Represents **HAS-A** relationship.
- **Child object** cannot exist without the parent.
- If parent is destroyed, child is **ALSO** destroyed.
- Represented by a filled diamond.



Real-world Example:

Car has Engine.

(Engine cannot exist without Car.

If Car is destroyed, Engine is destroyed)

UML Notation:



4. Comparison Summary

Feature	Association	Aggregation	Composition
Type	General Relationship	Weak HAS-A	Strong HAS-A
Dependency	Low (Independent)	Medium (Independent)	High (Dependent)
Child Lifecycle	Independent	Independent	Dependent on Parent
Parent Destroyed	No effect on child	No effect on child	Child is destroyed
Symbol	—	◇ (Hollow Diamond)	◆ (Filled Diamond)
Example	Teacher teaches Student	Team has Players	House has Rooms

5. Real-world Examples

• Association:

A Teacher teaches a Student.
(Teacher and Student can exist independently)



• Aggregation:

A Team has Players.
(Players can be part of another Team too)



• Composition:

A House has Rooms.
(Rooms cannot exist without House)



6. Code Examples (Java)

(A) Association

```

class Teacher {
    class Student {
        Teacher teacher; // Association
    }

    Student s = new Student();
    s.teacher = new Teacher();

    // Student and Teacher are independent
    // objects.
  
```

(B) Aggregation

```

class Department {
    List<Student> students; // Aggregation
}

Department dept = new Department();
dept.students = new ArrayList<>();
dept.students.add(new Student());

// Students can exist even if Department
// object is removed.
  
```

(C) Composition

```

class Engine { }
class Car {
    private Engine engine = new Engine(); // Composition
}

Car c = new Car();
// Engine cannot exist without Car.

// If Car object is destroyed, Engine
// also becomes unreachable.
  
```

7. Key Takeaways

- ✓ Association = general relationship.
- ✓ Aggregation = weak HAS-A.
- ✓ Composition = strong HAS-A.
- ✓ Use Aggregation when child can exist independently.
- ✓ Use Composition when child cannot exist without parent.
- ✓ Always identify the lifecycle dependency correctly.



Quick Memory Trick

- A = Association (A simple line)
- Ag = Aggregation (Hollow diamond)
- Co = Composition (Solid diamond)



Common Interview Questions

1. What is Association in OOP?
2. Difference between Aggregation and Association?
3. Difference between Aggregation and Composition?
4. When to use Aggregation and when Composition?



Remember

"Association = **Knows**"
Aggregation = **Has**"
Composition = **Owns**"



Choosing the right relationship makes your design more **maintainable** and **bug-free**!



(OBJECT ORIENTED PROGRAMMING)

★ SOLID PRINCIPLES ★

SOLID is a set of 5 design principles introduced by **Robert C. Martin (Uncle Bob)**. They are guidelines to help developers write **better, clean, and maintainable** code.



Interview Tip

SOLID Principles help us to write code that is more maintainable, flexible, scalable and easy to understand.

S Single Responsibility Principle (SRP)	Definition A class should have only one reason to change. It should have only one responsibility or job.	Java Example <pre>// Bad Example class User { void saveUser() { // save user to DB } void sendEmail() { // send email to user } } // Good: Separate classes</pre>	Real-world Analogy A Chef should only cook food. He shouldn't also deliver food and manage payments. 	Interview Tip ★ If a class is doing more than one job, any change in one job may affect other jobs. So, keep classes focused and small.
O Open/Closed Principle (OCP)	Definition Software entities (class, module, function) should be open for extension but closed for modification.	Java Example <pre>interface Shape { double area(); } class Circle implements Shape { public double area() { ... } } class Rectangle implements Shape { public double area() { ... } } // We can add new shapes without // modifying existing code</pre>	Real-world Analogy A software system should be easy to add new features without changing the existing working code. 	Interview Tip ★ Use inheritance, interfaces and abstraction to add new functionality instead of changing existing code.
L Liskov Substitution Principle (LSP)	Definition Subtypes must be substitutable for their base types without breaking the program behavior.	Java Example <pre>class Bird { void fly() { ... } } class Sparrow extends Bird { void fly() { ... } // OK } // Ostrich is not a Bird (can't fly) // So don't force inheritance.</pre>	Real-world Analogy You should be able to replace a child class object with its parent class object without breaking things. 	Interview Tip ★ If subclass cannot honor the behavior of parent class, inheritance is not the right choice. Follow "is-a" relationship.
I Interface Segregation Principle (ISP)	Definition Clients should not be forced to depend on interfaces they do not use. Keep interfaces small and specific.	Java Example <pre>// Bad: Fat Interface interface Worker { void work(); void eat(); void sleep(); } // Good: Segregated Interfaces interface Workable { void work(); } interface Eatable { void eat(); } interface Sleepable { void sleep(); }</pre>	Real-world Analogy Don't force a machine that only prints to also scan and fax. Give it only what it needs. 	Interview Tip ★ Create small, role-specific interfaces instead of one big interface for everything.
D Dependency Inversion Principle (DIP)	Definition High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces).	Java Example <pre>// Abstraction interface Database { void connect(); } // Low-level Module class MySQLDatabase implements Database { public void connect() { ... } } // High-level Module class UserService { private Database db; // depends on abstraction public UserService(Database db) { this.db = db; } } // We can inject any Database (MySQL, Oracle, etc.)</pre>	Real-world Analogy A Switch depends on an Abstraction (Electricity), not on a specific Bulb. You can plug in any bulb. 	Interview Tip ★ Depend on Abstractions, not on concrete implementations. Use Dependency Injection.

SOLID At a Glance



- S - Single Responsibility → One class, one job.
- O - Open/Closed → Open for extension, closed for modification.
- L - Liskov Substitution → Subtypes must be substitutable.
- I - Interface Segregation → Many small interfaces are better.
- D - Dependency Inversion → Depend on abstractions, not on details.

Benefits of SOLID

- ✓ Easy to understand
- ✓ Easy to maintain
- ✓ Easy to extend
- ✓ Reduces bugs
- ✓ Promotes good architecture



"SOLID principles are not rules, they are **GUIDELINES** to become a **better** developer."





IMPORTANT OOP INTERVIEW QUESTIONS



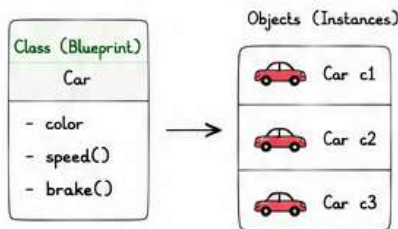
Interview Tip

Focus on Concepts, Examples and **Real-world use cases**.
Give short, clear and confident answers.

① What is the difference between Object and Class?

Ans:

- Class is a blueprint or template.
- Object is an instance of a class.



② What is the difference between Abstract Class and Interface?

Ans:

- Abstract class can have both abstract and concrete methods.
- Interface can have only abstract methods (before Java 8).
- Abstract class has state; Interface has no state.

Abstract Class	Interface
Can have constructors	Cannot have constructors
Can have non-abstract methods	Cannot have non-abstract methods (before Java 8)
Can have instance variables	Only public static final variables
Uses 'extends'	Uses 'implements'

③ What is the difference between Method Overloading and Method Overriding?

Ans:

- Overloading → Same method name, different parameters, same class.
- Overriding → Same method name, same parameters, different class (parent-child).

Overloading	Overriding
Compile-time (Static)	Runtime (Dynamic)
Same class	Parent & Child class
Different parameters	Same parameters
Return type can be different	Return type must be same or covariant

④ What is Encapsulation?

Ans:

- It is the wrapping of data (variables) and methods together into a single unit (class).
- It hides the internal details and shows only essential features.

```
class BankAccount {
    private double balance; // hidden data
    public void deposit(double amt) { ... }
    public double getBalance() { ... }
}
```

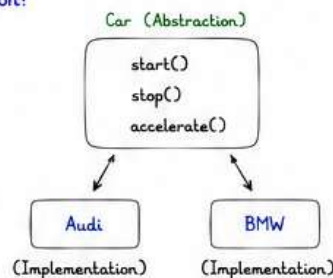
Access using getter/setter methods.



⑤ What is Abstraction?

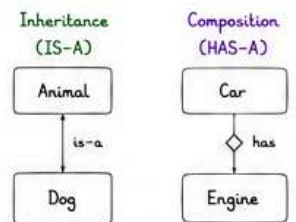
Ans:

- It shows only essential features and hides implementation details.
- Achieved using Abstract class or Interface.



⑥ What is the difference between Composition and Inheritance?

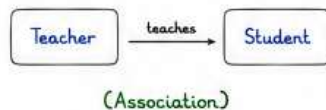
Inheritance (IS-A)	Composition (HAS-A)
Strong relationship	Stronger relationship
Child cannot exist without parent	Child can exist independently
Tight coupling	Loose coupling
Implemented using extends	Implemented using has-a (object inside class)



⑦ What is Association?

Ans:

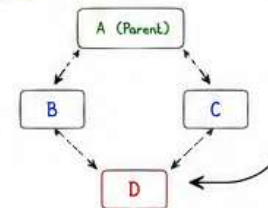
- It is a weak relationship between two objects.
- Both objects can exist independently.



⑧ What is the Diamond Problem?

Ans:

- When a class inherits from two classes that have a common parent, it creates ambiguity.
- Solved in Java using Interfaces (default) or virtual inheritance (in C++).



D gets confused about which version of members to use.

⑨ What is the difference between Constructor and Method?

Ans:

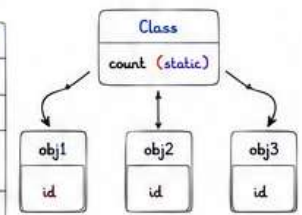
```
class Student {
    Student() {
        // constructor
    }
    void display() {
        // method
    }
}
```

Constructor	Method
Same name as class	Can have any name
No return type	Must have return type (or void)
Called automatically when object is created	Called explicitly
Used for initialization	Used for behavior

⑩ What is the difference between static and instance?

Ans:

Static (Class level)	Instance (Object level)
Belongs to class	Belongs to object
Access using class name	Access using object
One copy shared by all objects	Separate copy for each object
Example: static int count;	Example: int id;



⑪ What is the difference between final, finally and finalize?

final	finally	finalize()
Keyword	Block	Method
Used with variable, method or class	Used in exception handling	Called by GC before object is destroyed
Makes it constant or non-overridable	Always executes (except on JVM crash)	Deprecated in latest Java versions

⑫ Why is OOP important?

Ans:

- Improves code reusability
- Enhances maintainability
- Provides data security
- Supports real-world modeling
- Helps in large & complex applications
- Promotes better design (SOLID)



Good OOP Design
= Clean Code
= Happy Developer 😊



OOPs CONCEPTS

(OBJECT ORIENTED PROGRAMMING)

★ DESIGN PATTERNS - BASICS ★

Design Patterns are proven solutions to common problems in software design. They are not ready-made code, but **reusable** ideas and templates that make code more **flexible**, **scalable** and **maintainable**.



Interview Tip

Don't just remember the name. Understand the **purpose**, **when to use** and a simple real-world example.

1. SINGLETON

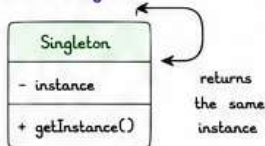
Purpose:

Ensure a class has only one instance and provide a global access point to it.

When to use:

For managing shared resources like configuration, logging, database connections.

UML / Diagram:



Example:

Logger, Cache, Thread Pool, Database Connection

2. FACTORY

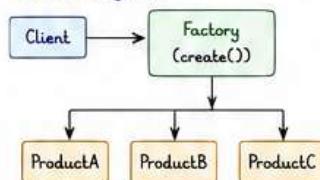
Purpose:

Define an interface for creating an object, but let subclasses decide which class to instantiate.

When to use:

When a class doesn't know which type of object it needs to create.

UML / Diagram:



Example:

Notification (Email, SMS, Push), Payment Gateway

3. BUILDER

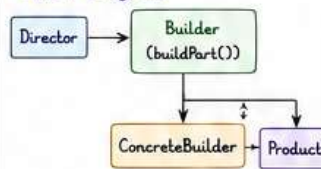
Purpose:

Separate the construction of a complex object from its representation.

When to use:

When an object has many optional parts or complex construction steps.

UML / Diagram:



Example:

StringBuilder, House Builder, Report Generator

4. STRATEGY

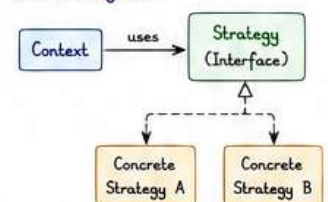
Purpose:

Define a family of algorithms, encapsulate each one, and make them interchangeable.

When to use:

When you want to switch between different algorithms at runtime.

UML / Diagram:



Example:

Discount Calculation, Sorting Algorithms, Payment Methods

5. OBSERVER

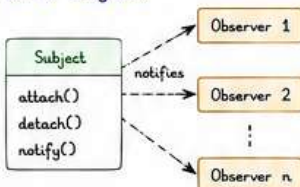
Purpose:

Define a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.

When to use:

When we need to notify multiple objects about changes in one object.

UML / Diagram:



Example:

Event Listeners, Stock Market Updates, Newsletter Subscription

6. ADAPTER

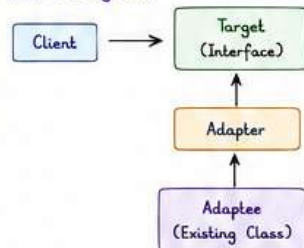
Purpose:

Convert the interface of a class into another interface clients expect.

When to use:

When you want to use a class but its interface doesn't match your requirement.

UML / Diagram:



Example:

Plug Adapter, Legacy System Integration, Third-party Libraries

7. DECORATOR

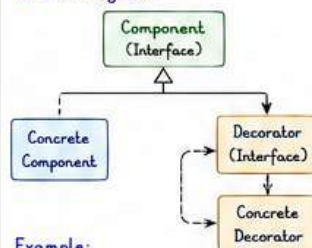
Purpose:

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing.

When to use:

When we want to add behavior to objects without modifying their structure.

UML / Diagram:



Example:

I/O Streams, GUI Components, Pizza (Extra Cheese, Toppings)

8. MVC (MODEL VIEW CONTROLLER)

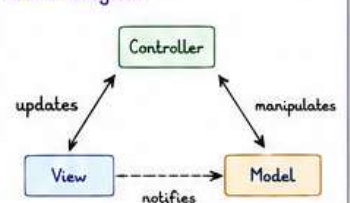
Purpose:

Separate an application into Model (data), View (UI), and Controller (logic).

When to use:

For building large applications with clear separation of concerns.

UML / Diagram:



Example:

Web Apps (Spring MVC), Android Apps, Desktop Applications

How to Remember?

S	Singleton	→ Only One Instance
F	Factory	→ Creates Objects
B	Builder	→ Builds Complex Objects
S	Strategy	→ Swap Algorithms
O	Observer	→ Observes Changes
A	Adapter	→ Adapts Interface
D	Decorator	→ Adds Responsibilities
M	MVC	→ Separates Concerns

Quick Comparison

Pattern	Type	Focus	Key Benefit
Singleton	Creational	Single instance	Global access, controlled instance
Factory	Creational	Object creation	Flexibility in object creation
Builder	Creational	Complex object construction	Readable, step-by-step creation
Strategy	Behavioral	Algorithm family	Interchangeable algorithms
Observer	Behavioral	Notifications	Loose coupling
Adapter	Structural	Interface conversion	Compatibility with existing code
Decorator	Structural	Add behavior	Extend without modifying
MVC	Architectural	App structure	Separation of concerns

General Interview Tips

- ✓ Always explain with a real-world example.
- ✓ Mention **when to use** and **when not to use**.
- ✓ Draw a simple diagram.
- ✓ Keep the answer short, clear and structured.



"Design Patterns are the building blocks of maintainable and scalable software."



These concepts help in writing robust, efficient and extensible code. They are widely used in large scale real-world applications.



Interview Tip

Advanced concepts show depth of your understanding. Use **real-world** examples whenever possible.

1. ABSTRACTION

Idea:

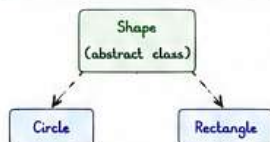
Show only essential features and hide implementation details.

How?

Using abstract classes and interfaces.

Example:

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }
}
```



2. INTERFACE vs ABSTRACT CLASS

Interface	Abstract Class
Only method signatures	Can have both methods + variables
Multiple inheritance is possible	Single inheritance only
All methods are public by default	Access modifiers are allowed
Variables are public static final	Variables can be any (static/non-static)

Interface Example:

```
interface Animal {
    void eat(); // public abstract
}
class Dog implements Animal {
    public void eat() {
        System.out.println("Eating");
    }
}
```

3. STATIC vs INSTANCE

Static (Class level)	Instance (Object level)
Belongs to class	Belongs to object
Memory allocated once	Memory allocated for each object
Accessed using class name	Accessed using object reference
Shared by all objects	Unique for each object

Example:

```
class Counter {
    static int count = 0; // shared
    int id; // unique
    Counter(int id) {
        this.id = id;
        count++;
    }
}
```

4. THIS vs SUPER

this:

- Refers to current object.
- Used to access current class members.

```
class Student {
    String name;
    Student(String name) {
        this.name = name; // this -> current obj
    }
}
```

super:

- Refers to parent class object.
- Used to access parent class members.

```
class Animal {
    void sound() { System.out.println("Sound"); }
}
class Dog extends Animal {
    void sound() {
        super.sound(); // call parent method
        System.out.println("Bark");
    }
}
```

5. INNER / NESTED CLASSES

A class defined inside another class.

Types:

- Static Nested Class
- Non-static Inner Class
- Local Inner Class
- Anonymous Inner Class

Example:

```
class Outer {
    int x = 10;
    class Inner {
        void show() {
            System.out.println(x); // can access
        }
    }
}
```

6. ENUM (ENUMERATION)

Used to define a group of constants.

Example:

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY
}
class Test {
    Day today = Day.MONDAY;
}
```

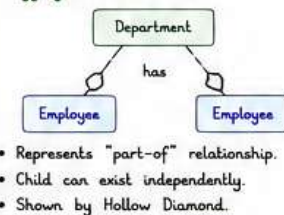
Benefits:

- Type safe
- Improves readability
- Easy comparison



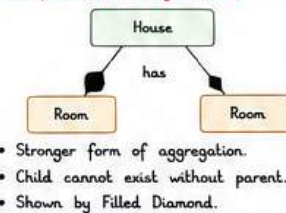
7. AGGREGATION vs COMPOSITION

Aggregation (Weak HAS-A)



- Represents "part-of" relationship.
- Child can exist independently.
- Shown by Hollow Diamond.

Composition (Strong HAS-A)



- Stronger form of aggregation.
- Child cannot exist without parent.
- Shown by Filled Diamond.

8. COUPLING vs COHESION

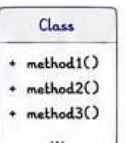
Coupling (Between classes)

- Measure of dependency between classes.
- Aim for Loose Coupling. (Less dependency is good)



Cohesion (Within a class)

- Measure of how closely related methods are within a class.
- Aim for High Cohesion. (More related is good)



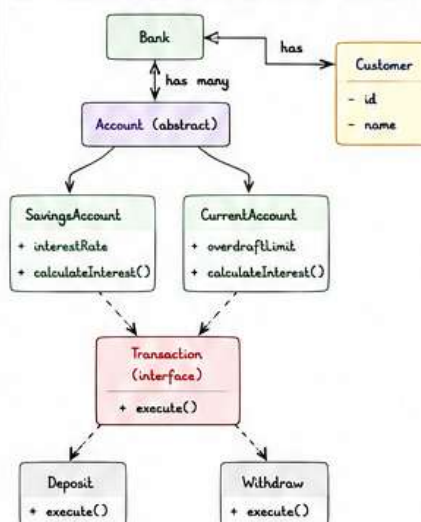
9. ACCESS MODIFIERS (VISIBILITY)

Modifier	Same Class	Same Package	SubClass (Diff Package)	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

10. KEYWORDS QUICK RECAP

class	extends	implements
Defines a class	Inherit a class	Implement an interface
new	this	super
Create new object	Current object reference	Parent class reference
static	final	abstract
Belongs to class	Cannot be changed/overridden	Incomplete class or method

11. REAL-WORLD EXAMPLE (BANK SYSTEM)



12. KEY TAKEAWAYS

- ✓ Abstraction hides complexity.
- ✓ Encapsulation protects data.
- ✓ Inheritance promotes reusability.
- ✓ Polymorphism provides flexibility.
- ✓ Design patterns solve common problems.
- ✓ High cohesion and low coupling lead to better design.
- ✓ Always choose "is-a" (inheritance) or "has-a" (composition) wisely.

Common Interview Questions

1. Difference between Abstraction and Encapsulation?
2. When to use Abstract Class and when Interface?
3. this vs super?
4. Difference between Aggregation and Composition?
5. Why static is used?

